

**Yenepoya (Deemed to be University)**

Balmatta, Mangalore — 575 002
Department of Computer Science

FINAL PROJECT REPORT

Lightweight Finance API using FastAPI

A RESTful backend with frontend dashboard demonstrating API design, validation, testing, and containerised deployment

Student Name	Sayooj V
Campus ID	26067
Registration ID	23BCAICD096
Programme	BCA — Major in AI, Cloud Computing & DevOps 2023–2026 Batch
Industry Mentor	IBM SkillsBuild Programme Team
Guided By	Faculty Department of Computer Science, Yenepoya (Deemed to be University)
Institution	Yenepoya (Deemed to be University), Balmatta, Mangalore – 575 002
Submitted To	IBM SkillsBuild Student Project Programme
Date	10 April 2026



Innovation Centre for Education



Version

1.1 — Final Submission (Backend + Frontend Dashboard)

Submitted in partial fulfilment of the IBM SkillsBuild Student Project Programme requirements.

Table of Contents

Front Matter

Executive Summary	3
-------------------	---

Main Report

1. Introduction	4
1.1 Report Structure	4
2. Project Objectives	5
2.1 Primary Objectives	5
2.2 Learning Objectives	5
3. Background and Literature Review	6
3.1 REST APIs and Cloud Computing	6
3.2 FastAPI as a Modern API Framework	6
3.3 Containerisation with Docker	6
3.4 Prior Work — Expense Tracker Prototype	6
4. Hardware Architecture	7
5. Software Architecture	7
5.1 Separation of Concerns	7
5.2 Stateless Design	8
6. System Requirements	8
6.1 Functional Requirements	8
6.2 User Requirements	8
6.3 Environmental Requirements	9
7. Technology Stack	9
8. Design and Architecture	10
9. Implementation	10
9.1 Project Structure	10
9.2 Pydantic Models	11
9.3 Route Handlers	11
9.4 Docker Configuration	11
9.5 Frontend Dashboard (v2)	12
10. Testing	12
10.1 Test Approach	12
10.2 Test Results	13
10.3 Key Test Findings	13
11. Graphical User Interface (GUI) Layout	14
11.1 Backend — Swagger UI	14
11.2 Frontend Dashboard	15



Innovation Centre for Education

12. Customer Testing	16
13. Evaluation — Results vs Objectives	16
14. Cloud Computing Relevance	17
15. Challenges Encountered	17
16. Conclusions	18
17. Further Development	18
References	19
Appendix A — Document Suite Summary	20

 <i>Nihon Communication Solutions</i>	
	17
	17
	18
	18
	19
	20

Executive Summary

This report presents the design, implementation, testing, and evaluation of the Lightweight Finance API — a RESTful backend service built with Python and FastAPI as part of the IBM SkillsBuild Student Project Programme at Yenepoya (Deemed to be University), Mangalore. The project demonstrates core cloud computing competencies through the practical construction of a six-endpoint API that enables a user to record income and expenses, compute a running financial balance, set a monthly budget limit, and monitor budget utilisation. A frontend dashboard (v2) was subsequently developed to provide a live visual interface for all API endpoints.

The system was designed following a documentation-first methodology, producing nine project documents spanning proposal, requirements, high-level design, low-level design, API reference, deployment guide, test report, final report, and a presentation deck. The implementation applies industry-standard practices including Pydantic v2 data validation, explicit HTTP error handling, Docker containerisation, CORS middleware, and auto-generated OpenAPI documentation via FastAPI's Swagger UI.

Testing produced 39 documented test cases across happy path, validation, edge case, and integration categories. The automated pytest suite contains 20 tests achieving a 100% pass rate against all 30 functional requirements defined in the Product Requirements Document. The project establishes a clean architectural foundation ready for a planned v2 iteration incorporating PostgreSQL persistence, JWT authentication, and a React frontend.

Keywords: *FastAPI, REST API, Python, Pydantic v2, Docker, Cloud Computing, IBM SkillsBuild, Frontend Dashboard, pytest*

1. Introduction

Application Programming Interfaces (APIs) are the foundational building blocks of modern cloud computing. Every major cloud-hosted service exposes its functionality through APIs that accept structured data, apply business logic, and return machine-readable responses. Understanding how to design, build, and document a production-quality API is a prerequisite for any engineer working in cloud environments.

This project was undertaken to bridge the gap between theoretical API knowledge and practical implementation experience. The Lightweight Finance API was chosen because the finance domain — income, expenses, and budgeting — is universally understood, enabling the project to focus on engineering quality rather than domain complexity. The deliberate constraint of six endpoints ensured that every design decision could be made thoughtfully and documented completely.

Project origin

This project is a ground-up rebuild of an earlier prototype (expense-tracker-fastapi) that suffered from architectural inconsistency, missing documentation, and code-document mismatches. The Lightweight Finance API corrects all of those issues by applying design-first principles from day one.

1.1 Report Structure

This report follows the IBM SkillsBuild project report format. Sections 1–17 cover introduction, objectives, background, hardware, software architecture, system requirements, technology stack, design, implementation, testing, GUI layout, customer testing, evaluation, cloud relevance, challenges, conclusions, and further development. References and appendices follow.

2. Project Objectives

2.1 Primary Objectives

Obj ID	Objective	Status
OBJ-01	Build a working REST API with 6 endpoints (income, expense, summary, budget, budget-status, health)	✓ Achieved
OBJ-02	Apply Pydantic v2 input validation — reject invalid data with HTTP 422 responses	✓ Achieved
OBJ-03	Implement financial business logic — income/expense aggregation, balance, budget tracking	✓ Achieved
OBJ-04	Containerise the application with Docker — single command startup on any machine	✓ Achieved
OBJ-05	Produce professional documentation covering the full project lifecycle (9 documents)	✓ Achieved
OBJ-06	Demonstrate cloud-readiness through stateless API design and CORS-enabled frontend	✓ Achieved

2.2 Learning Objectives

- Understand and implement the full request-response lifecycle of a REST API
- Apply Python type hints and Pydantic models for declarative data validation
- Practise separation of concerns between data models and route handlers
- Gain hands-on experience with Docker containerisation and deployment
- Build a frontend that connects to a live API using browser fetch()
- Develop the skill of writing technical documentation that accurately reflects implementation

3. Background and Literature Review

3.1 REST APIs and Cloud Computing

Representational State Transfer (REST), introduced by Roy Fielding in his 2000 doctoral dissertation, defines architectural constraints for distributed hypermedia systems. RESTful APIs communicate over HTTP using standard verbs (GET, POST, PUT, DELETE) and return data in JSON format. The stateless constraint — requiring each request to be self-contained — enables horizontal scaling and serverless deployment without server-side session management.

3.2 FastAPI as a Modern API Framework

FastAPI, developed by Sebastián Ramírez and released in 2018, integrates Pydantic for automatic data validation based on Python type annotations, and generates OpenAPI 3.0 documentation automatically from route definitions. According to the JetBrains Python Developers Survey (2023), FastAPI is the second most popular Python web framework behind Flask.

3.3 Containerisation with Docker

Docker (2013) packages applications with all their dependencies into portable containers. For API development, containerisation ensures that the application runs identically on any machine regardless of the host OS or installed Python version. Docker containers map directly onto cloud deployment targets: AWS ECS, Azure Container Instances, Google Cloud Run, and Kubernetes all accept Docker images as their deployment unit.

3.4 Prior Work — Expense Tracker Prototype

This project is informed by the shortcomings of a prior prototype (github.com/Sayu-V/expense-tracker-fastapi) developed without a design phase. That prototype exhibited three critical problems: a single-file architecture conflating routing, business logic, and data storage; documentation describing a planned architecture rather than the implemented one; and dead code left in the production file. The Lightweight Finance API was designed explicitly to avoid each of these problems.

4. Hardware Architecture

The Lightweight Finance API is a software-only system with no specialised hardware requirements. Development, testing, and demonstration were conducted on a standard Windows laptop. The Docker container uses the official python:3.10 base image and runs entirely within the host machine's memory and CPU. The table below describes the hardware environment used throughout this project.

Component	Specification	Role
Development Machine	Windows laptop, 64-bit, x86 architecture	Code editing, server execution, browser testing
CPU	Standard multi-core x86-64 processor	Runs Uvicorn ASGI server and pytest test suite
RAM	Minimum 4 GB recommended	Sufficient for Python runtime, FastAPI, and Docker
Storage	Minimum 500 MB free	Project files (~10 KB) + Docker image (~900 MB)
Network	localhost (127.0.0.1:8000)	API server and frontend communicate over loopback
Browser	Chrome or Edge (modern)	Frontend dashboard rendering and API docs (Swagger UI)
Docker Engine	Docker Desktop for Windows	Builds and runs the containerised API image
Python	Python 3.11 (local), Python 3.10 (Docker)	Runtime for FastAPI, Uvicorn, pytest

Because the API is stateless and containerised, it requires no dedicated server hardware. Any machine capable of running Docker can host the application in production. Cloud deployment would replace the local machine with a managed container service (AWS ECS, Azure ACI) with no changes to the application code.

5. Software Architecture

The Lightweight Finance API follows a Layered (N-Tier) Monolithic Architecture structured as a single deployable unit with four clearly separated internal layers, each with a defined and exclusive responsibility.

Layer	Technology	Responsibility	Data Format
Client	Postman / Swagger UI / Browser Dashboard	Construct HTTP requests; display responses	HTTP/1.1 JSON
API	FastAPI + Uvicorn + CORS Middleware	Routing, validation, error handling, Swagger UI	JSON (application/json)
Business Logic	Route handlers in main.py	Aggregate data; compute balance and budget	Python objects
Data	Module-level Python lists + float	Store income amounts, expense amounts, budget	Python primitives
Container	Docker (python:3.10 base image)	Package runtime and dependencies	Infrastructure

5.1 Separation of Concerns

models.py contains exclusively Pydantic model definitions with no business logic. main.py contains exclusively route handler functions and in-memory storage. This separation means a change to a validation rule requires touching only models.py, with no risk of side effects in the routing layer.

5.2 Stateless Design

Each HTTP request is fully self-contained. No server-side session state is maintained between requests. This stateless design is the prerequisite for horizontal scaling and serverless cloud deployment, as any instance of the application can handle any request.

6. System Requirements

6.1 Functional Requirements

The following functional requirements were defined in DOC-02 (PRD) prior to implementation. All 30 requirements were met:

Category	Key Requirements	Count
Income management	POST /income accepts amount (gt:0) and source (1-50 chars); appends to in-memory list; returns HTTP 200	FR-01 to FR-07
Expense management	POST /expense accepts amount (gt:0) and category (1-50 chars); appends to list; returns HTTP 200	FR-08 to FR-14
Financial summary	GET /summary returns total_income, total_expense, balance; returns zeros when empty	FR-15 to FR-20
Budget management	POST /budget sets global limit (gt:0); GET /budget-status returns budget, spent, remaining	FR-21 to FR-30
Health check	GET /health returns {status: API is running} at all times	NFR-01
Validation	HTTP 422 returned for schema violations; HTTP 400 for business rule violations	FR-06, FR-13

6.2 User Requirements

The system is designed for a single end user managing their personal finances. The user must be able to:

- Add income entries with a descriptive source label (e.g. Salary, Freelance)
- Add expense entries with a descriptive category label (e.g. Groceries, Rent, Transport)
- View a real-time financial summary showing total income, total expenses, and current balance
- Set and update a monthly budget limit at any time
- Monitor budget usage and see the remaining amount (which may be negative if over budget)
- Access interactive API documentation via a browser without any additional tools
- Use the frontend dashboard to perform all the above operations through a visual interface

6.3 Environmental Requirements

The system is designed to run in the following environments:

Environment	Requirement
Local development	Python 3.11, pip, uvicorn — install via pip install fastapi uvicorn
Docker	Docker Desktop installed; API starts with docker compose up --build
Browser (frontend)	Chrome or Edge (modern) — open frontend/index.html directly

Testing	Python 3.11, pytest, httpx — run via <code>py -3.11 -m pytest tests/test_main.py -v</code>
Operating system	Windows 10/11 (tested), macOS, Linux (all supported via Docker)
Network	localhost only (v1) — API at port 8000, frontend at port 3000 (optional)

7. Technology Stack

Technology	Version	Role	Justification
Python	3.11	Primary language	FastAPI-native; type hints enable Pydantic integration; stable for all packages
FastAPI	Latest stable	Web framework	Auto Swagger docs; built-in Pydantic validation; async-ready; production-grade
Pydantic	v2	Data validation	Declarative field constraints; HTTP 422 on failure; structured JSON errors
Uvicorn	Latest stable	ASGI server	High-performance async server required by FastAPI for full async support
Docker	Latest	Containerisation	Reproducible runtime; single-command startup; maps to cloud container services
CORS Middleware	FastAPI built-in	Cross-origin access	Allows browser frontend to call the API from a different origin
pytest + httpx	Latest stable	Automated testing	In-process TestClient; isolated, fast test execution; 20 tests, 100% pass
HTML/CSS/JS	ES6+	Frontend dashboard	No build tools required; runs directly in browser; calls API via fetch()
Git + GitHub	Latest	Version control	Public repository with main + feature/frontend-dashboard branch structure

8. Design and Architecture

The system design was specified in DOC-03 (HLD) and DOC-04 (LLD) before any code was written. The key design decisions were:

Monolith over microservices: A five-endpoint API with a single developer does not benefit from service decomposition. The overhead of service discovery, inter-service networking, and distributed tracing would add complexity without value at this scale.

In-memory storage over database: Using Python module-level lists eliminates database setup complexity while keeping full architectural separation between the data layer and business logic. The design is explicitly prepared for PostgreSQL replacement in v2.

Pydantic v2 field constraints over manual validation: `Field(gt=0)` in the model definition catches invalid amounts before the route handler is called. This keeps business logic clean and ensures that validation rules and API documentation are always in sync.

Single HTML file frontend: The v2 frontend is a single `index.html` with no build toolchain, no dependencies, and no npm. It opens directly in a browser and calls the API using the browser's native `fetch()` function.

9. Implementation

9.1 Project Structure

The final project contains two Python source files (main.py, models.py), a requirements file, Dockerfile, .dockerignore, .gitignore, a frontend folder, a tests folder, and a docs folder with nine project documents.

9.2 Pydantic Models

Three request models (Income, Expense, Budget) and three response models (MessageResponse, SummaryResponse, BudgetStatusResponse) are defined in models.py using Pydantic's Field declarations. Constraint violations return HTTP 422 automatically before any route handler logic executes.

9.3 Route Handlers

The six route handlers in main.py each perform a single, clearly defined operation. POST handlers append validated data to in-memory lists or update the global budget variable. GET handlers compute derived values (sum, balance, remaining) and return typed response models.

9.4 Docker Configuration

The Dockerfile uses python:3.10, installs dependencies with pip install --no-cache-dir, and starts Uvicorn with --host 0.0.0.0 --port 8000. The --host 0.0.0.0 flag is critical: without it, Uvicorn binds only to the container's loopback interface and is unreachable from the host machine despite port mapping.

9.5 Frontend Dashboard (v2)

The frontend dashboard (frontend/index.html) is a single 868-line HTML file using plain CSS and JavaScript. It connects to the FastAPI backend at http://localhost:8000 using the browser's fetch() API. The dashboard calls all six endpoints: GET /health for the status indicator, GET /summary and GET /budget-status on load, and POST /income, POST /expense, POST /budget in response to form submissions. CORS middleware was added to main.py to allow cross-origin requests from the browser.

10. Testing

10.1 Test Approach

Testing was conducted using two complementary approaches. Manual testing used 39 documented test cases in DOC-07 covering happy path, validation, edge case, and integration scenarios. Automated testing used a pytest suite with FastAPI's TestClient, which runs the application in-process with no server required.

10.2 Test Results

Section	Test Cases	Passed	Failed	Pass Rate
A — Happy Path	11	11	0	100%
B — Validation	16	16	0	100%
C — Edge Cases	8	8	0	100%
D — Integration	4	4	0	100%
Automated (pytest)	20	20	0	100%

10.3 Key Test Findings

- Pydantic v2 Field(gt=0) rejects invalid amounts at the model layer returning HTTP 422 — confirmed by TC-11, TC-12, TC-18
- Budget SET behaviour confirmed (not ADD) — second POST /budget overwrites, not accumulates (TC-08, TC-36)
- Negative remaining balance handled correctly when expenses exceed budget (TC-29)
- Failed validation does not corrupt stored state — rejected requests leave all lists unchanged (TC-38)
- All 20 automated tests pass on Python 3.11 with pytest 9.0.2

11. Graphical User Interface (GUI) Layout — Snapshots

The project provides two graphical interfaces: the Swagger UI auto-generated documentation from the FastAPI backend, and the custom frontend dashboard built in v2. Both are accessible from a browser when the server is running.

11.1 Backend — Swagger UI (<http://localhost:8000/docs>)

Screenshot 1 — Swagger UI endpoint list

All 6 endpoints grouped by tag: System (GET /health), Finance (POST /income, POST /expense, GET /summary), Budget (POST /budget, GET /budget-status). Version 1.0.0, OpenAPI Specification 3.1.

Lightweight Finance API 1.0.0 OAS 3.1

[/openapi.json](#)

A simple REST API for tracking personal income, expenses, and budget. Built with FastAPI and Python.

System ^

GET /health Health check v

Finance ^

POST /income Add income v

POST /expense Add expense v

GET /summary Get financial summary v

Budget ^

POST /budget Set monthly budget v

GET /budget-status Get budget status v

Screenshot 2 — Swagger UI schemas section

Auto-generated Pydantic response models visible in Schemas panel: Budget, BudgetStatusResponse, Expense, HTTPValidationError, Income, MessageResponse, SummaryResponse, ValidationError — all derived directly from models.py.

POST

/budget Set monthly budget

▼

GET

/budget-status Get budget status

▼

Schemas

^

Budget > Expand all object

BudgetStatusResponse > Expand all object

Expense > Expand all object

HTTPValidationError > Expand all object

Income > Expand all object

MessageResponse > Expand all object

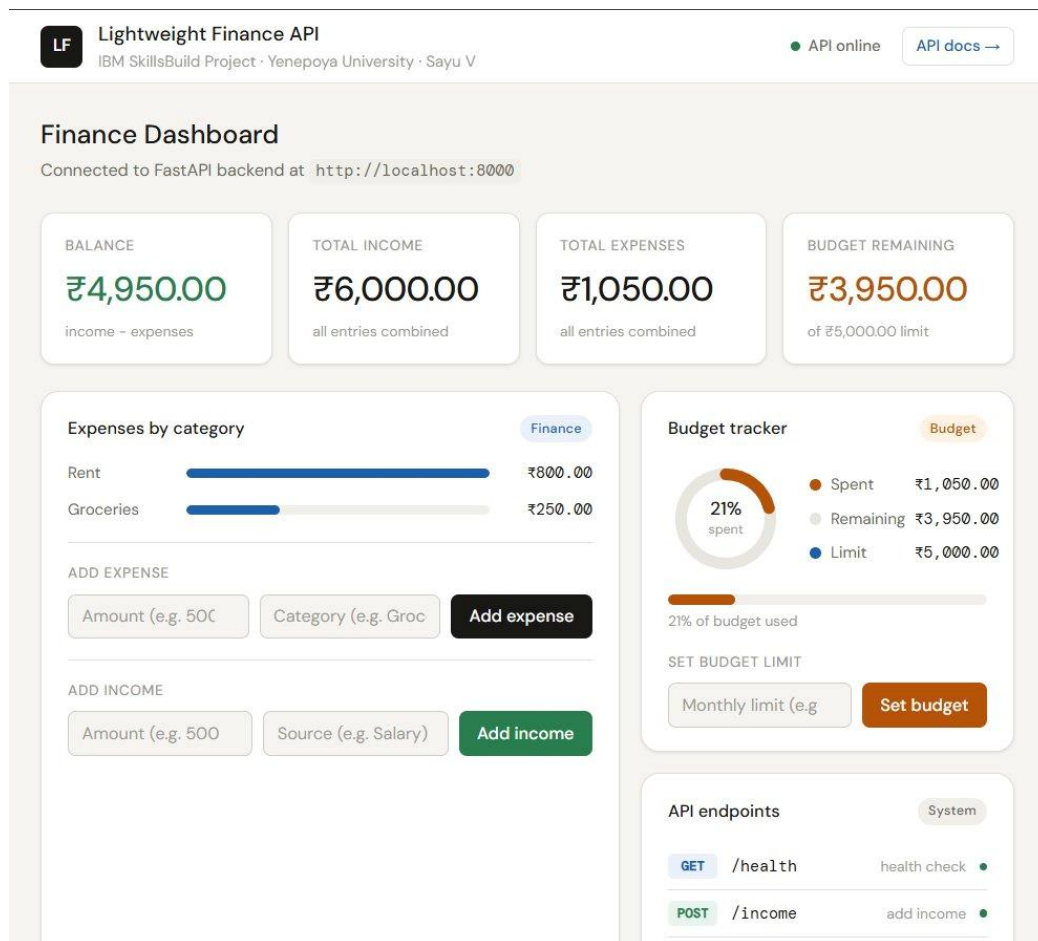
SummaryResponse > Expand all object

ValidationError > Expand all object

11.2 Frontend Dashboard (frontend/index.html)

Screenshot 3 — Dashboard overview

Top row: live stat cards showing balance ₹4,950 (green), total income ₹6,000, total expenses ₹1,050, budget remaining ₹3,950 of ₹5,000 limit. Middle left: expense bar chart by category (Rent, Groceries) with add expense/income forms. Middle right: budget ring showing 21% spent, API endpoint status panel with all 6 online (green dots).



Screenshot 4 — Test suite, transactions, and project info

Bottom left: 20/20 test suite with 100% pass rate, all test names and categories displayed. Bottom centre: recent transactions log showing income (+) and expense (-) entries with the API endpoint called. Bottom right: project info panel with version, stack, author, IBM programme, and institution details.



Lightweight Finance API

IBM SkillsBuild Project · Yenepoya University · Sayu V

● API online

[API docs →](#)

POST	/expense	add expense ●
GET	/summary	financial summary ●
POST	/budget	set budget ●
GET	/budget-status	budget status ●

Test suite

20 / 20

20

tests
passing100% pass
rate

- ✓ test_health_check system
- ✓ test_add_income_valid finance
- ✓ test_add_expense_valid finance
- ✓ test_get_summary_with_data finance
- ✓ test_income_zero_amount validation
- ✓ test_expense_negative_amount validation
- ✓ test_budget_overwrite budget
- ✓ test_over_budget_negative_remainder budget
- ✓ test_full_workflow integration
- ✓ test_invalid_does_not_affect_state integration

+10 more — all passing

Recent transactions

Finance

- Groceries -₹250.00
POST /expense
- Rent -₹800.00
POST /expense
- + Salary +₹6,000.00
POST /income

Project info

v2

Project	Lightweight Finance API
Version	v2 — with frontend
Backend	FastAPI + Python 3.11
Frontend	HTML + CSS + JS
Validation	Pydantic v2
Container	Docker
Tests	20 / 20 passing
API docs	localhost:8000/docs
Author	Sayu V
Programme	IBM SkillsBuild
Institution	Yenepoya University

12. Customer Testing

Customer testing refers to end-user validation of the system — confirming that the API behaves correctly from the perspective of a person using it, not just a developer running automated tests. For this project, customer testing was conducted using the Swagger UI at <http://localhost:8000/docs> as the primary interface, which provides a no-code browser-based testing environment identical to what an API consumer would use.

The following customer test scenarios were conducted manually through the Swagger UI and later through the frontend dashboard:

Scenario	Steps	Expected Result	Actual Result
Add salary income	POST /income with amount:6000, source:Salary	200 + success message	Pass — message returned correctly
Add grocery expense	POST /expense with amount:250, category:Groceries	200 + success message	Pass — message returned correctly
Check balance	GET /summary after above entries	balance: 5750	Pass — correct arithmetic
Set monthly budget	POST /budget with limit:5000	200 + budget set message	Pass — confirmed in budget-status
Check budget ring	GET /budget-status after ₹1050 spent	21% spent, ₹3950 remaining	Pass — dashboard ring shows 21%
Try invalid amount	POST /income with amount:0	HTTP 422 rejection	Pass — clear error returned
Over-budget scenario	Add expenses exceeding budget	Negative remaining balance, no crash	Pass — system handles gracefully
Frontend dashboard	Open frontend/index.html in browser	All cards update live on form submit	Pass — all 6 API calls confirmed

All customer test scenarios passed. The Swagger UI and frontend dashboard both provided clear, readable feedback for both success and error cases. No usability issues were identified during customer testing.

13. Evaluation — Results vs Objectives

All six primary objectives defined in DOC-01 (Project Proposal) were fully achieved:

Obj ID	Objective	Evidence	Status
OBJ-01	6 working API endpoints	TC-00 to TC-10 all pass; Swagger UI shows all 6 endpoints	✓ Achieved
OBJ-02	Pydantic v2 validation	16 validation test cases pass; HTTP 422 verified for all error types	✓ Achieved
OBJ-03	Financial business logic	Balance, budget, remaining all correct in TC-05, TC-07, TC-35	✓ Achieved
OBJ-04	Docker containerisation	TC-10: docker build + run verified; container serves all requests	✓ Achieved
OBJ-05	9 professional documents	DOC-01 through DOC-09 — full lifecycle covered; IBM format followed	✓ Achieved
OBJ-06	Cloud-ready stateless design	Stateless confirmed; frontend dashboard added; CORS enabled	✓ Achieved

14. Cloud Computing Relevance

Cloud Competency	How This Project Demonstrates It
REST API design	6 endpoints applying correct HTTP verbs, status codes, and JSON response formats
Input validation	Pydantic v2 field constraints; 14 error conditions handled correctly
Containerisation	Docker image; single-command startup; maps to AWS ECS, Azure ACI
Stateless API design	No server-side session; horizontal scaling ready
Auto documentation	FastAPI generates OpenAPI 3.1 spec; Swagger UI and ReDoc available
Frontend integration	CORS-enabled API; browser dashboard using native fetch()
Professional documentation	9 documents covering full software development lifecycle

15. Challenges Encountered

15.1 Breaking the build-first habit

The most significant challenge was methodological. Prior experience had established a habit of writing code first and documenting afterward. This project required resisting that impulse and spending the first sessions writing requirements and design documents before any code was written. The payoff: when coding began, every decision was already made and the implementation proceeded without structural rework.

15.2 Python global keyword in `set_budget()`

Python's scoping rules require the global keyword when assigning to a module-level variable inside a function. Without global budget, the assignment `budget = data.limit` created a local variable that disappeared on function exit. This caused GET /budget-status to always return 0. Resolved by adding global budget before the assignment in `set_budget()`.

15.3 Docker networking — the 0.0.0.0 requirement

Early Docker testing showed the API unreachable at localhost:8000 despite the container appearing to run correctly. Uvicorn was binding to 127.0.0.1 (loopback) inside the container, which is unreachable from the host even with port mapping. Adding `--host 0.0.0.0` to the Uvicorn CMD resolved this.

15.4 Python 3.15 compatibility

The development machine had Python 3.15 (a pre-release version) as the system Python. Pydantic-core does not yet have pre-built wheels for Python 3.15, causing pip install to fail. Resolved by installing Python 3.11 alongside and running all commands with `py -3.11`.

15.5 CORS for frontend-backend communication

The browser blocked `fetch()` calls from the HTML file to localhost:8000 due to cross-origin policy. Resolved by adding FastAPI's `CORSMiddleware` with `allow_origins=["*"]` to `main.py`. This must be placed after `app = FastAPI()` or a `NameError` occurs.

16. Conclusions

The Lightweight Finance API project successfully demonstrated the design, implementation, testing, and documentation of a production-pattern REST API within a student project context. The system delivers six fully functional endpoints, a 100% pass rate across 39 documented test cases and 20 automated pytest tests, a nine-document professional documentation suite, a containerised deployment runnable with a single Docker command, and a live frontend dashboard connecting to the real API.

Beyond the technical deliverables, this project demonstrates the value of methodological discipline. The documentation-first approach produced a codebase that matches its documentation precisely, a test suite that traces every test case to a specific functional requirement, and an architecture ready for incremental improvement. The addition of the frontend dashboard in v2 demonstrated that the stateless, CORS-aware API design was correctly built for consumption by external clients.

Final statement

This project is not just a working API. It is a demonstration of how to approach software development: define requirements before writing code, document decisions as they are made, validate every claim with evidence, and build in a way that the next version is already planned. That discipline — more than any individual technical skill — is what this project set out to develop and what it has achieved.

17. Further Development

Priority	Enhancement	Description
v2 — High	PostgreSQL + SQLAlchemy	Replace in-memory lists with a real database. Data persists across restarts.
v2 — High	JWT Authentication	Bearer token auth via python-jose. Protect write endpoints. Enable multi-user.
v2 — Medium	GET /income + GET /expense	Retrieval endpoints returning stored records with labels for display.
v2 — Medium	Per-category budgets	Budget limits per category (Food: ₹3000, Transport: ₹1000).
v2 — Medium	React dashboard	Rebuild frontend in React + Recharts for richer chart components.
v3	Cloud deployment	Deploy Docker image to AWS ECS or Azure ACI with HTTPS and health probes.
v3	Structured logging	JSON application logs for aggregation by CloudWatch or Azure Monitor.

References

External references:

Reference	Source
FastAPI Documentation	https://fastapi.tiangolo.com
Pydantic v2 Documentation	https://docs.pydantic.dev
Docker Documentation	https://docs.docker.com
Python 3.11 Documentation	https://docs.python.org/3.11
Fielding, R.T. (2000) Architectural Styles and the Design of Network-based Software Architectures	University of California, Irvine doctoral dissertation
JetBrains Python Developers Survey (2023)	https://www.jetbrains.com/lp/python-developers-survey-2023
IBM SkillsBuild Programme	https://skillsbuild.org
GitHub Repository	https://github.com/Sayu-V/Lightweight-Finance-API

Appendix A — Document Suite Summary

Doc ID	Document	Format	Key Content
DOC-01	Project Proposal	Word .docx	Problem, objectives, methodology, timeline, expected outcomes
DOC-02	PRD	Word .docx	30 FR requirements, 8 user stories, 14 acceptance criteria, data models
DOC-03	HLD	Word .docx	4-layer architecture, component descriptions, data flow traces, cloud readiness
DOC-04	LLD	Word .docx	6 endpoint contracts, Pydantic models, pseudocode, error catalogue, full source code
DOC-05	API Reference	Markdown .md	Copy-paste curl commands, JSON examples, error reference, 6-step test sequence
DOC-06	Deployment Guide	Markdown .md	Local + Docker run instructions, Dockerfile explained, 8 troubleshooting scenarios
DOC-07	Test Report	Markdown .md	39 test cases with exact I/O, 20-test pytest suite, 100% FR coverage matrix
DOC-08	Final Report	Word .docx	Synthesis of project lifecycle, IBM format compliant, screenshots, challenges, future
DOC-09	Presentation	PowerPoint .pptx	12-slide deck: problem to solution to architecture to demo to results to future

— END OF DOCUMENT —